MDP Case Studies

How R(s,a) and P(s’|s,a) enter Chess, Poker, and GridWorld

Week 5 — Reinforcement Learning: The Idea

From Conway to LangGraph · Dipartimento di Fisica, UniBo

The Two Quantities That Must Be Defined in Every MDP

The Bellman optimality equation for Q*

$$Q^*(s,a) = R(s,a) + \gamma \Sigma_{s'} P(s'|s,a) \cdot \max_{a'} Q^*(s',a')$$

R(s,a) — The reward model

Signal the environment gives the agent after taking action a in state s.

Can be:

- Dense: non-zero at every step
- Sparse: zero almost everywhere, non-zero only at terminal states
- Mixed: small per-step cost + large terminal reward

P(s'|s,a) — The transition model

Distribution over next states given s and action a.

Can be:

- Deterministic: exactly one s' with probability 1
- Stochastic: a distribution over multiple s'
- Hidden: agent cannot observe the true s (POMDP)

Q-learning is model-free: it never needs to write down R or P explicitly. Instead it observes the reward r directly, and the environment samples s' from P automatically when the agent takes action a. The table is filled in by experience, not by specification.

♟ Chess — Deterministic Transitions, Sparse Terminal Reward

MDP components

S	Board configuration — positions of all 32 pieces (up to ~10^{43} states)
A	Legal moves (~20 on average, up to 218 in one position)
R	+1 win · −1 loss · 0 draw — emitted only at the final move
Markov?	YES — the board encodes the full game state; no hidden information

P(s'|s,a) — deterministic, but huge

Given board s and legal move a, exactly one board position s' results. The sum ΣP[...] collapses to a single term. Q-learning observes s' directly without needing to enumerate ~10^{43} states.

R(s,a) — sparse terminal reward

For all non-terminal moves:

$$R(s,a) = 0$$

For the final move only: R = +1, −1, or 0.
Bellman for non-terminal s:

$$Q^*(s,a) = \gamma \max_{a'} Q^*(s',a')$$

Key difficulty: the agent must play a full game (~40 moves) before receiving any reward signal. The +1 or −1 at the end propagates backward through γ^t discounting. With $\gamma = 0.99$ and $t = 40$ moves, the first move receives $\gamma^{40} \approx 0.67$ of the terminal reward. Sparse reward is the hardest case for Q-learning.

♥ Poker — Partial Observability Breaks the Markov Property

MDP components

S	Visible cards + own hand (opponent's hand is hidden)
A	Fold, call, raise
R	Chips won/lost at showdown (terminal reward only)
Markov?	NO — opponent's cards are hidden; observed state is incomplete

P(s'|s,a) — ill-defined from the agent's perspective

The true game state is:

(own hand, opp. hand, board cards)

The agent observes only:

(own hand, board cards)

P(s'|s_obs, a) now integrates over all possible opponent hands consistent with the observation. This is a POMDP (Partially Observable MDP).

R(s,a) — sparse, high variance

Chips won/lost at showdown only. Same optimal strategy can produce wildly different outcomes due to card randomness.

The POMDP solution: the agent must maintain a belief state — a probability distribution over all full game states consistent with what it has observed. The agent acts on this belief state rather than the observed state. Standard Q-learning cannot be applied directly; it requires extensions (e.g. Q-learning over belief states, or CFR for poker).

CFR — Counterfactual Regret Minimization è l'algoritmo che ha risolto il Texas Hold'em a due giocatori (Libratus, 2017; Pluribus, 2019).

GridWorld — The Pedagogically Clean Case

MDP components

S	(row, col) — 5×5 = 25 states, each fully observable
A	Up, Down, Left, Right (4 discrete actions)
R	−0.01 per step + +1.0 at goal cell (3,4)
Markov?	YES — current position fully describes the state

Move Right from (r,c):
P(s'|s,a) — deterministic, one non-zero term

P((r,c+1)|(r,c), Right) = 1 if (r,c+1) is free

P((r,c) |(r,c), Right) = 1 if blocked by wall

The code does this implicitly:

```
if valid: self.state = (nr, nc)

else:     self.state unchanged
```

R(s,a) — mixed dense + terminal
Step cost −0.01 penalises long paths.
Q-learning observes s' without knowing P.

Terminal +1.0 rewards reaching the goal.

Why the step cost −0.01 is essential: without it, every path to the goal is equally valued (pure terminal reward = Chess). With it, $Q^*(s, a_opt) \approx \gamma^d \cdot 1 - 0.01 \cdot (1-\gamma^d)/(1-\gamma)$ for a cell at distance d. The term penalising length makes the greedy policy extract the shortest path automatically.

Summary Comparison: R, P, and Q-Learning Applicability

	♟ Chess	♥ Poker	▦ GridWorld
P(s'|s,a)	Deterministic (one s' per move)	Stochastic + partially hidden (POMDP)	Deterministic (move or bounce)
Sum ΣP[...]	Single term (collapses to one s')	Integral over hidden cards (ill-defined from obs.)	Single term (collapses to one s')
R(s,a)	±1 at game end only (pure sparse)	±chips at showdown (sparse + high variance)	−0.01 every step +1 at goal (mixed)
Reward type	Sparse terminal	Sparse terminal	Dense + terminal
Markov?	YES (board = full state)	NO (opponent hand hidden)	YES (position = full state)
Tabular Q-learning	Intractable (~10^{43} states)	Not applicable (non-Markovian)	**✓ Exact** **(25 × 4 = 100 entries)**
Real-world approach	AlphaZero (MCTS + neural net)	CFR / belief-state deep RL	Q-learning (this notebook!)

GridWorld: Where P(s'|s,a) and R(s,a) Live in the Code

The env.step(action) method implements both P and R simultaneously

```
def step(self, action):

    nr, nc = r+dr, c+dc


    # P(s'|s,a): transition model

    if valid(nr,nc) and not wall:

        self.state = (nr,nc)   # P=1

    # else: unchanged  # P=1 bounce


    # R(s,a): reward model
```

P(s'|s,a) is implicit:
The if/else block is the transition function. The environment never stores a matrix P — it executes the physics. Q-learning reads s' from self.state.

R(s,a) is conditional:

r=−0.01 every non-goal step; r=+1.0 at the goal. The Bellman equation accumulates these via discounted return.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,] \quad \leftarrow \text{r and s' come from env.step(a)}$$

The Q-learning agent uses r and s' directly — R and P never appear explicitly

```
else:

    return state, -0.01, False
```

Why R(s,a) = −0.01 per Step Is Not Optional

Without the step cost: pure terminal reward (like Chess)

$R(s,a) = 0$ for all non-goal states $\Rightarrow$ $Q^*(s,a) = \gamma^d$ for any path of length d to the goal

Every path to the goal has the same Q-value. The greedy policy is arbitrary — there is no incentive to take the shortest route. The agent may wander and still converge to a 'correct' but suboptimal policy.

With the step cost: shortest-path incentive built into the Bellman equation

$R(s,a) = -0.01$ per step $\Rightarrow$ $Q^*(s, a_opt) \approx \gamma^d \cdot 1 - 0.01 \cdot (1-\gamma^d)/(1-\gamma)$

The second term grows with path length d. A longer path means more −0.01 penalties. The greedy policy therefore selects the action that minimises d — the shortest path emerges automatically from the Bellman equation.

Pure terminal reward (γ=0.9, d=6)

$Q^* = 0.9^6 = 0.531$

$Q^* = 0.9^3 = 0.729$ (d=3)

⇒ shorter path preferred... but only weakly via γ

With step cost (γ=0.9, d=6)

$Q^* \approx 0.531 - 0.063 = 0.468$

$Q^* \approx 0.729 - 0.029 = 0.700$ (d=3)

⇒ explicit penalty for length: shorter ≫ longer